

Supplementary Evidence Package for a Resilient External Key Lifecycle Model in Critical Infrastructure

Ofir Joseph^{a,*}, Itzhak Aviv^{a,b}, Eran Dahan^b, Irit Hadar^a

^a*Department of Information Systems, University of Haifa, 199 Abba Khoushy Avenue, Mount Carmel, Haifa 3498838, Israel*

^b*Institute for Cryptoeconomics, WU Vienna University of Economics and Business, Welthandelsplatz 1, Vienna 1020, Austria*

Abstract

This supplementary package preserves the deployment, workflow, device, lifecycle, analytical, and traceability evidence supporting the accompanying manuscript. The package distinguishes architectural reasoning, implemented prototype behavior, direct appliance observations, configured policy values, executable lifecycle validation, derived quantities, and analytical projections. Preserved materials include prototype topology, QKD appliance connectivity, EKM and VPN decomposition, executed access and storage workflows, direct device values, complete reachable state exploration, seeded state and transition sensitivity, parameterized source capacity tables, and claim evidence mappings. Every released record protects secret key material, credentials, reusable tokens, private addressing, and sensitive operational configuration while retaining the state relationships required for scientific interpretation.

Keywords: quantum key distribution, cloud key management, critical infrastructure, lifecycle validation, supplementary evidence

1. Evidence package purpose and classification

This supplementary package accompanies the article entitled “A Resilient External Key Lifecycle Model for QKD Supported Cloud Key Management in Critical Infrastructure.” The materials follow the manuscript evidence sequence from system context through workflow realization, source observation, lifecycle validation, analytical interpretation, and transfer qualification. This ordering supports reproducible interpretation while keeping screenshots, equations, executable models, and operational claims within their appropriate evidentiary roles.

Table 1 defines the evidence classes applied consistently across the complete package. The classification assigns cumulative device counters toward count history, analytical capacity maps toward modeled projections, and workflow screenshots toward functional sequence evidence, while primitive conformance remains attached to the production profile.

*Corresponding author.

Email address: joseph.ofir@gmail.com (Ofir Joseph)

Table 1: The evidence classification scheme preserves distinct interpretations for reasoning, implementation, observation, validation, and analytical projection.

Evidence class	Status label	Interpretive role
Architectural reasoning	Reasoned	Supports comparative placement, responsibility allocation, and critical service dependency interpretation.
Implemented prototype behavior	Functionally observed	Supports realized component interaction across the VPN and managed storage workflows.
Direct QKD appliance observation	Direct	Supports the captured source rate, QBER, inventory, and cumulative appliance counters.
Configured policy value	Configured	Defines rotation, refresh, activation, and continuity variables used by the prototype.
Executable lifecycle specification	Executable validation	Supports internal consistency across every reachable state within declared finite domains.
Derived mathematical quantity	Derived	Supports transparent unit conversion and arithmetic interpretation of direct appliance values.
Analytical scenario projection	Analytical	Supports source feasibility, interruption endurance, depletion onset, refill behavior, and transfer qualification.

2. Prototype environment and trust boundary evidence

The prototype combined a commercial QKD transmitter and receiver pair, dedicated optical connectivity, cloud hosted EKM and VPN services, Google Cloud managed storage, authenticated ingress, and one external client. QKD1 served cloud side components, while QKD2 supplied matching material toward the external VPN client. The quantum channel remained inside the physical devices and dedicated fiber, while application key delivery used reachable service interfaces.

Figure 1 preserves the original logical prototype view from the implementation campaign. The image locates cloud resources, the key management path, the external key manager, the VPN service, the external client, and paired QKD endpoints within one case topology.

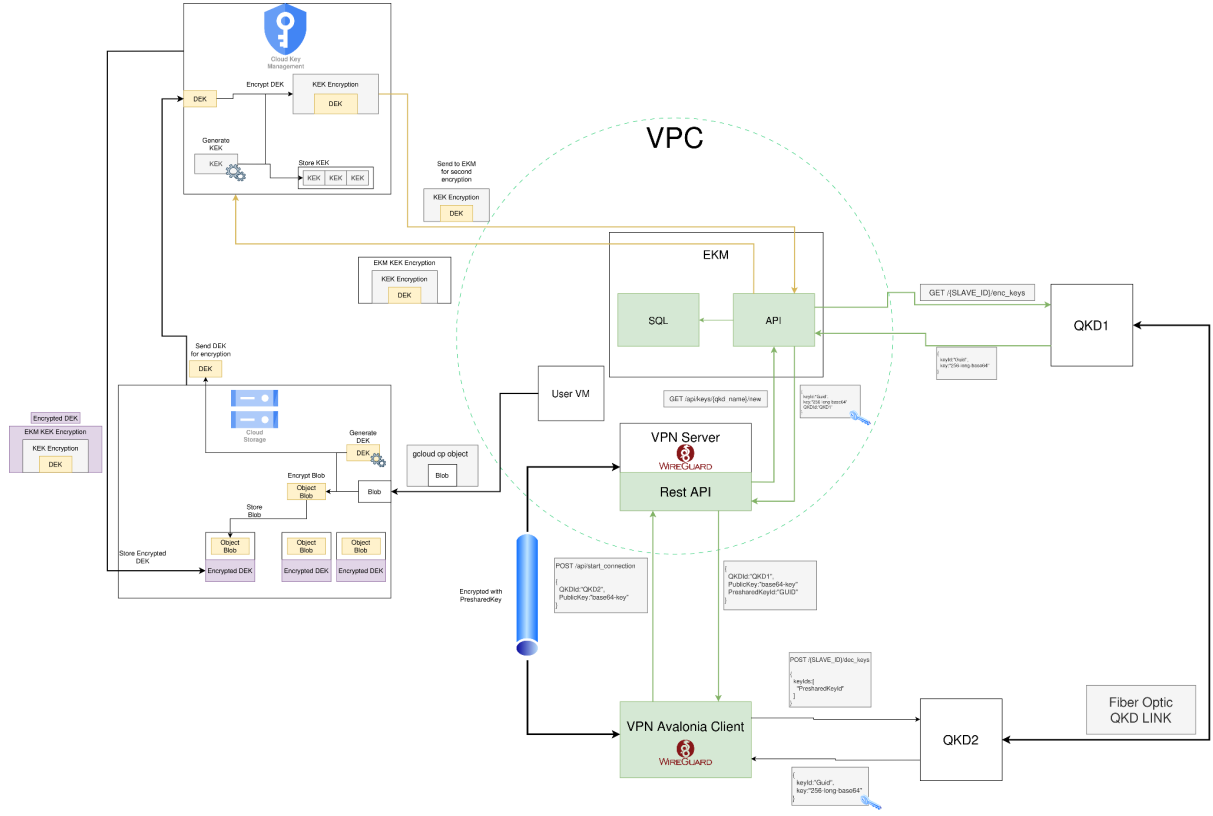


Figure 1: The original logical topology records managed resources, mediation services, the external client, and paired QKD endpoints used throughout the prototype case.

2.1. QKD appliance connectivity

Figure 2 records the physical QKD appliance connectivity used throughout the prototype. The quantum link and service facing interfaces remain visibly separated, supporting interpretation of physical key generation, management communication, and application delivery as distinct dependencies.

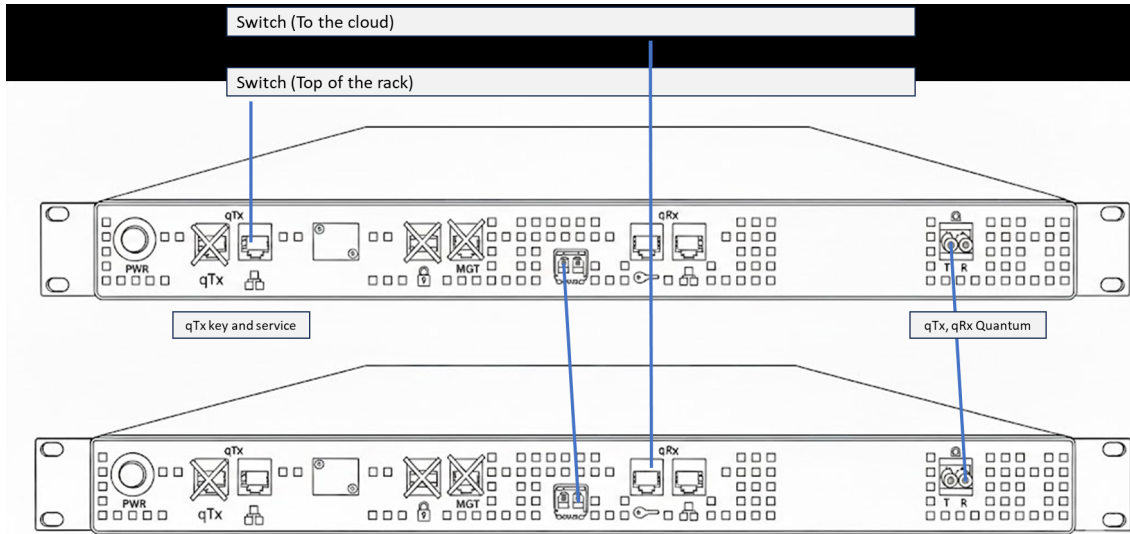


Figure 2: The QKD connectivity view distinguishes the dedicated quantum channel from service interfaces supporting management and application key delivery.

2.2. External key manager and VPN control services

Figure 3 preserves the implemented EKM decomposition used during the managed storage workflow. The service contains authenticated request handling, persistent consumed identifier records, temporary pooling, scheduled TTL cleanup, QKD1 retrieval, and external key processing. Persistent records retain authoritative identifier relationships after consumption, while temporary entries remain eligible for controlled allocation.

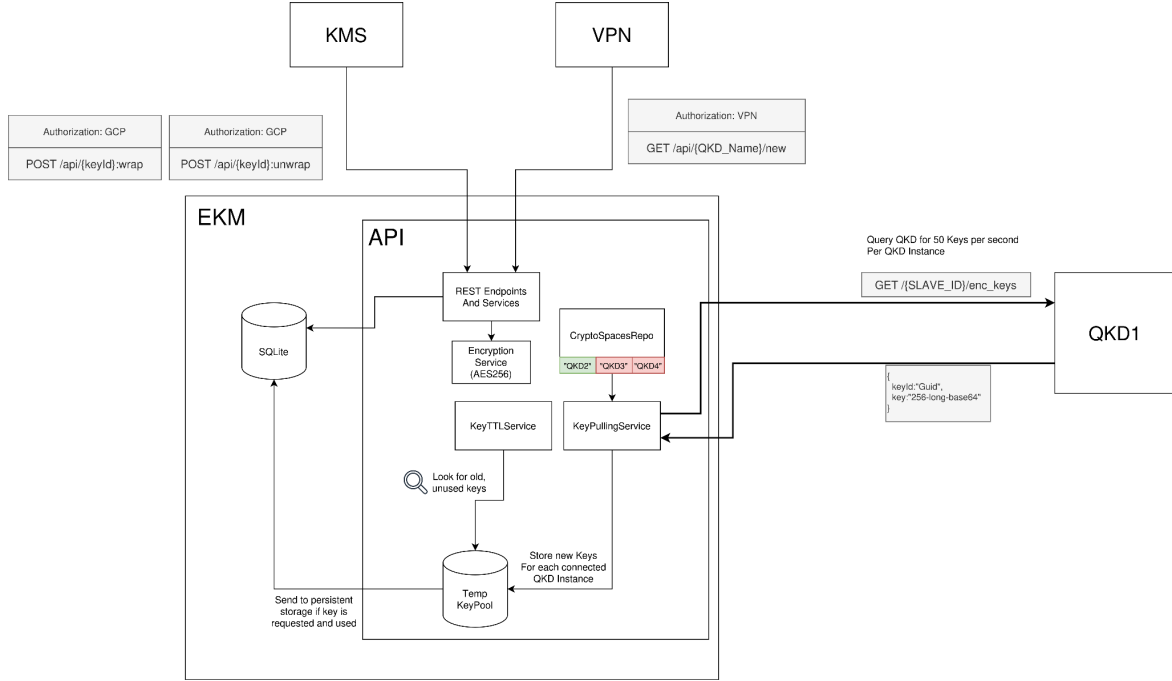


Figure 3: The EKM decomposition records authenticated handling, temporary pooling, QKD retrieval, TTL cleanup, and persistent consumed identifier relationships.

Figure 4 preserves the VPN service decomposition used during the access workflow. The VPN manager coordinates WireGuard peer state, while authenticated control endpoints return QKD identifiers and effective activation metadata. This arrangement preserves WireGuard packet processing while surrounding refresh with QKD specific lifecycle coordination.

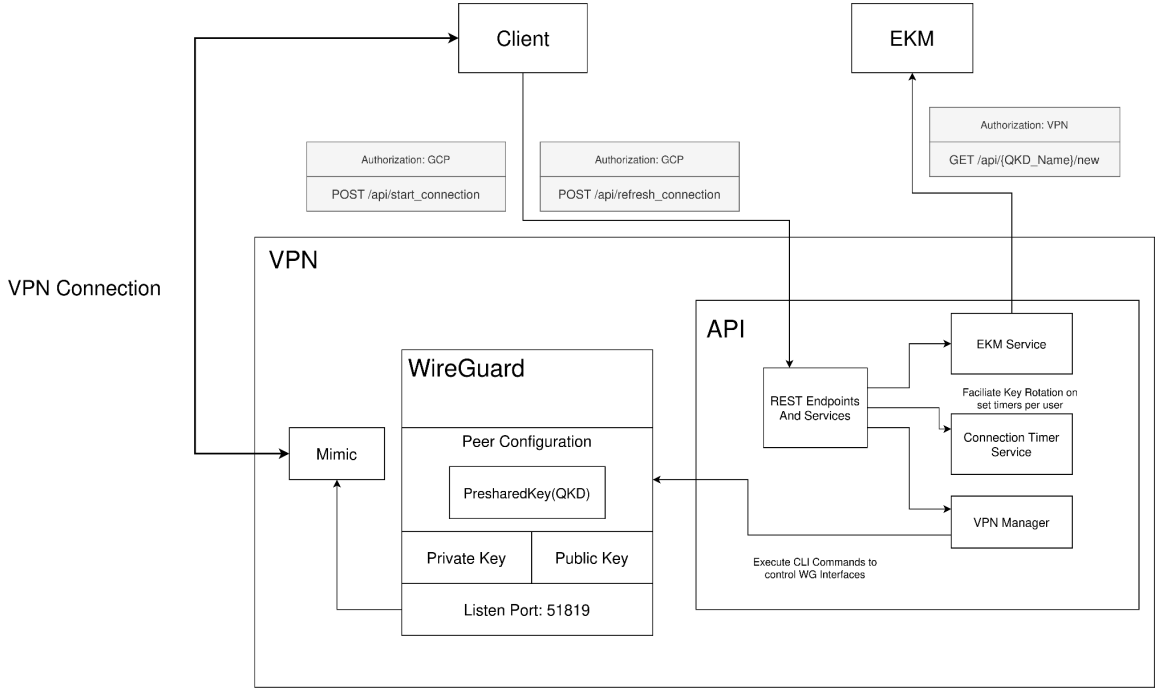


Figure 4: The VPN control service coordinates QKD identifiers and activation metadata while preserving WireGuard as the established tunnel mechanism.

3. Executed workflow evidence

3.1. QKD supported VPN access workflow

The VPN workflow exercised client authentication, identity aware reachability, interface preparation, server allocation, identifier return, matching QKD2 retrieval, WireGuard configuration, connected state, and coordinated refresh behavior. The preserved evidence records identifiers and lifecycle events while excluding secret key material.

Table 2: The VPN workflow connects observed client and server interactions with the lifecycle properties evaluated by the access scenario.

Workflow step	Observed event	Lifecycle interpretation
Authenticated ingress	The client authenticates and reaches the VPN service through identity aware mediation.	Access authentication remains separate from QKD allocation and activation state.
Server allocation	The VPN service requests an available QKD identifier through the external lifecycle path.	Allocation assigns one identifier toward the VPN purpose and session context.
Identifier return	The server returns the selected identifier and connection meta-data toward the authenticated client.	Both endpoints receive one shared reference supporting matched retrieval.
Client retrieval	The client obtains corresponding material from the paired QKD2 interface.	Identifier equality connects paired appliance delivery with the selected session context.
Tunnel activation	The client and server configure WireGuard using existing endpoint keys and the QKD supported PSK.	QKD material augments established WireGuard authentication and tunnel protection.
Coordinated re-fresh	The control path returns a new identifier and effective activation time.	Freshness and timing coordination govern transition toward the subsequent PSK state.

Figure 5 preserves a faithful redacted transcription of the client session together with its associated control trace. Figure 6 connects the same transcribed event sequence with the complete prototype component arrangement used during evaluation.

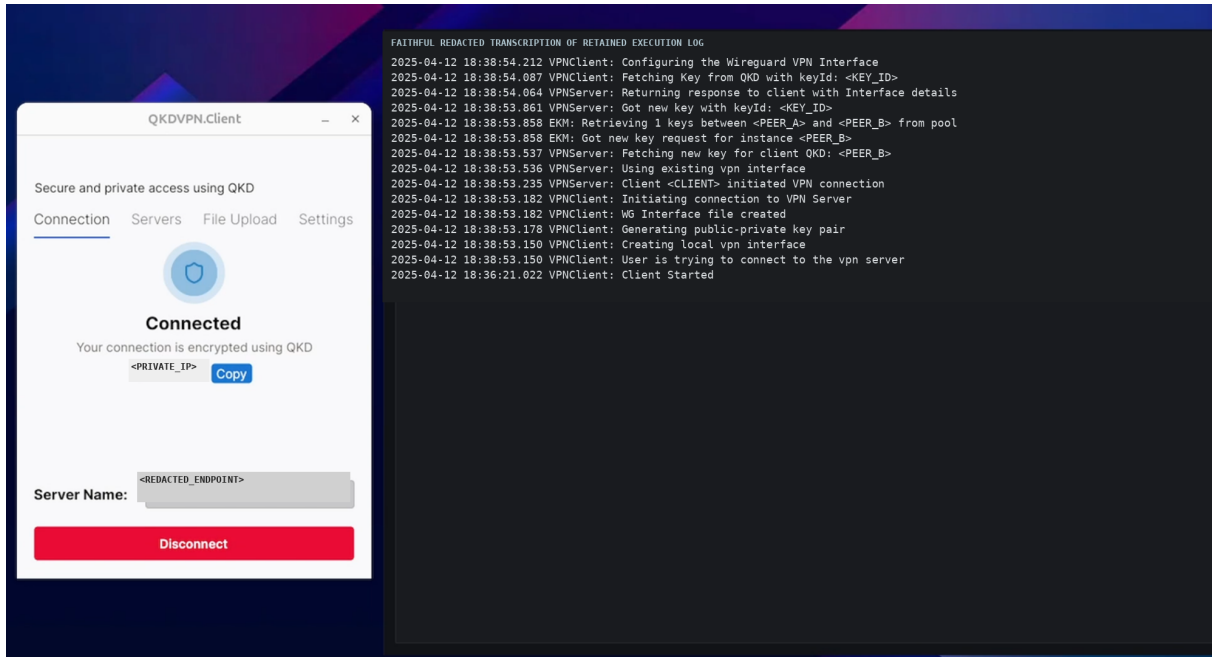


Figure 5: The faithful redacted VPN record preserves the retained timestamps and event order for interface preparation, identifier retrieval, server assignment, and nominal completion.

```

FAITHFUL REDACTED TRANSCRIPTION OF RETAINED EXECUTION LOG
2025-04-12 18:38:54.212 VPNClient: Configuring the Wireguard VPN Interface
2025-04-12 18:38:54.087 VPNClient: Fetching Key from QKD with keyId: <KEY_ID>
2025-04-12 18:38:54.064 VPNServer: Returning response to client with Interface details
2025-04-12 18:38:53.861 VPNServer: Got new key with keyId: <KEY_ID>
2025-04-12 18:38:53.858 EKM: Retrieving 1 keys between <PEER_A> and <PEER_B> from pool
2025-04-12 18:38:53.858 EKM: Got new key request for instance <PEER_B>
2025-04-12 18:38:53.537 VPNServer: Fetching new key for client QKD: <PEER_B>
2025-04-12 18:38:53.536 VPNServer: Using existing vpn interface
2025-04-12 18:38:53.235 VPNServer: Client <CLIENT> initiated VPN connection
2025-04-12 18:38:53.182 VPNClient: Initiating connection to VPN Server
2025-04-12 18:38:53.182 VPNClient: WG Interface file created
2025-04-12 18:38:53.178 VPNClient: Generating public-private key pair
2025-04-12 18:38:53.150 VPNClient: Creating local vpn interface
2025-04-12 18:38:53.150 VPNClient: User is trying to connect to the vpn server
2025-04-12 18:36:21.022 VPNClient: Client Started

```

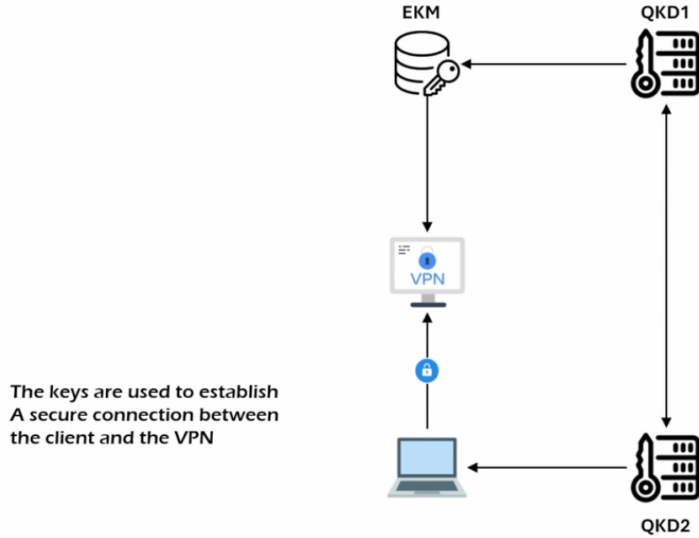


Figure 6: The faithful redacted VPN topology connects the retained control trace with the client, VPN service, EKM, and paired QKD endpoints.

3.2. Managed storage and external key workflow

The managed storage workflow separates three protection layers that require independent interpretation. Client side file protection demonstrates one optional application layer, managed storage retains provider generated data encryption keys according to provider documentation, and the KMS to EKM path governs external key operations. The retained execution log directly records an EKM key wrap request, successful upload processing, and subsequent object retrieval. The lower diagrams illustrate documented provider semantics and remain explicitly classified as architectural annotations, while the retained execution log supplies direct request evidence.

Table 3: The storage evidence distinguishes optional client protection, provider managed encryption, and externally governed key processing.

Protection layer	Preserved evidence	Supported interpretation
Optional client protection	Client trace records local QKD supported file protection before upload.	Supports application layer protection within the executed workflow.
Provider managed encryption	Provider documentation and the configured resource relationship define provider generated data key responsibility, while the retained log records successful upload through the configured workflow.	Supports envelope encryption responsibility separation and request path conformance at the managed service boundary.
External key processing	The retained EKM log records a key wrap request containing a redacted external key identifier, while the accompanying schematic illustrates expected wrapped data key return semantics.	Supports observed arrival of an external key request and architectural interpretation of the selected custody boundary.
Object persistence and retrieval	Storage commands record listing, transfer, and local inspection after cloud processing.	Supports completion of the executed storage workflow while assigning primitive conformance toward the production profile.

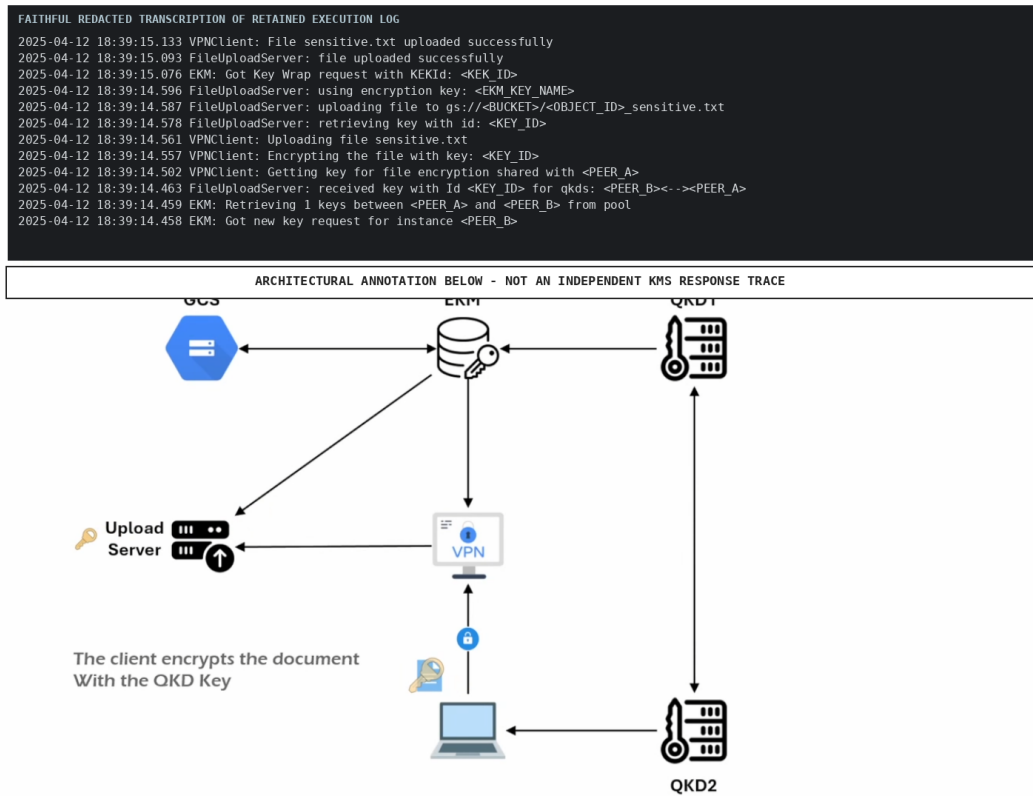


Figure 7: The faithful redacted client record preserves retained timestamps and events for local protection and upload initiation as an application layer separate from managed envelope encryption.

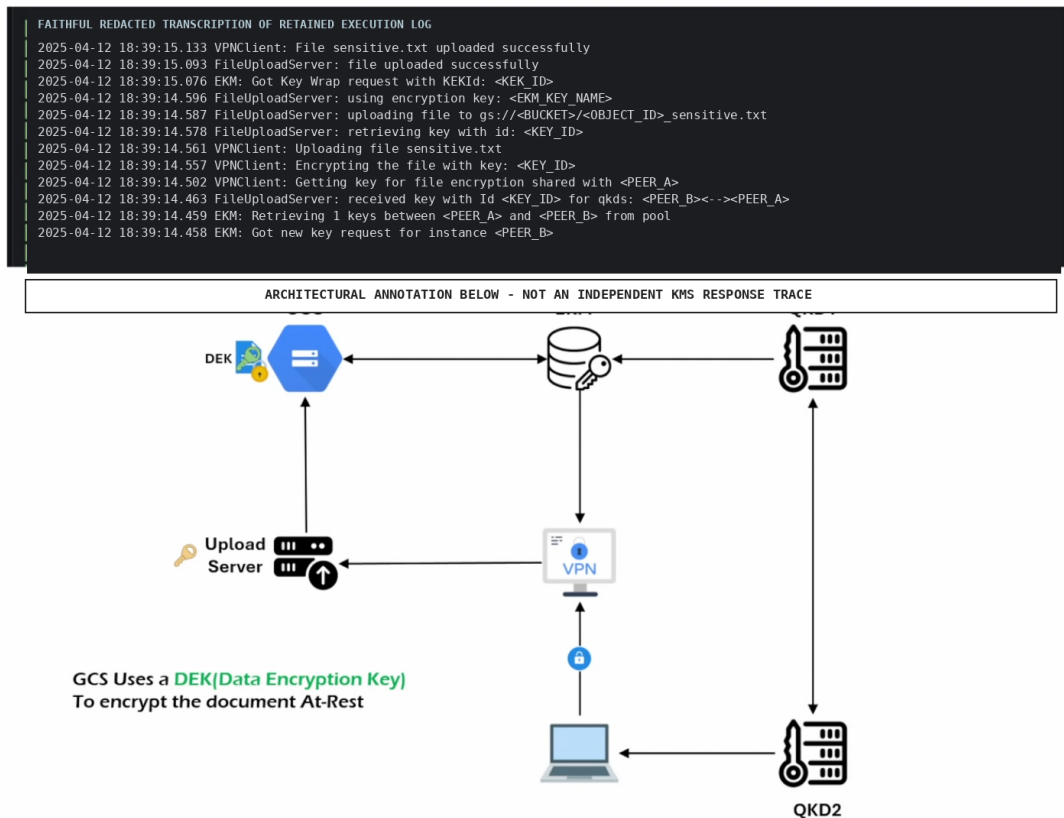


Figure 8: The faithful redacted log records the EKM key wrap request and successful upload, while the lower architectural annotation illustrates documented provider data key responsibility.

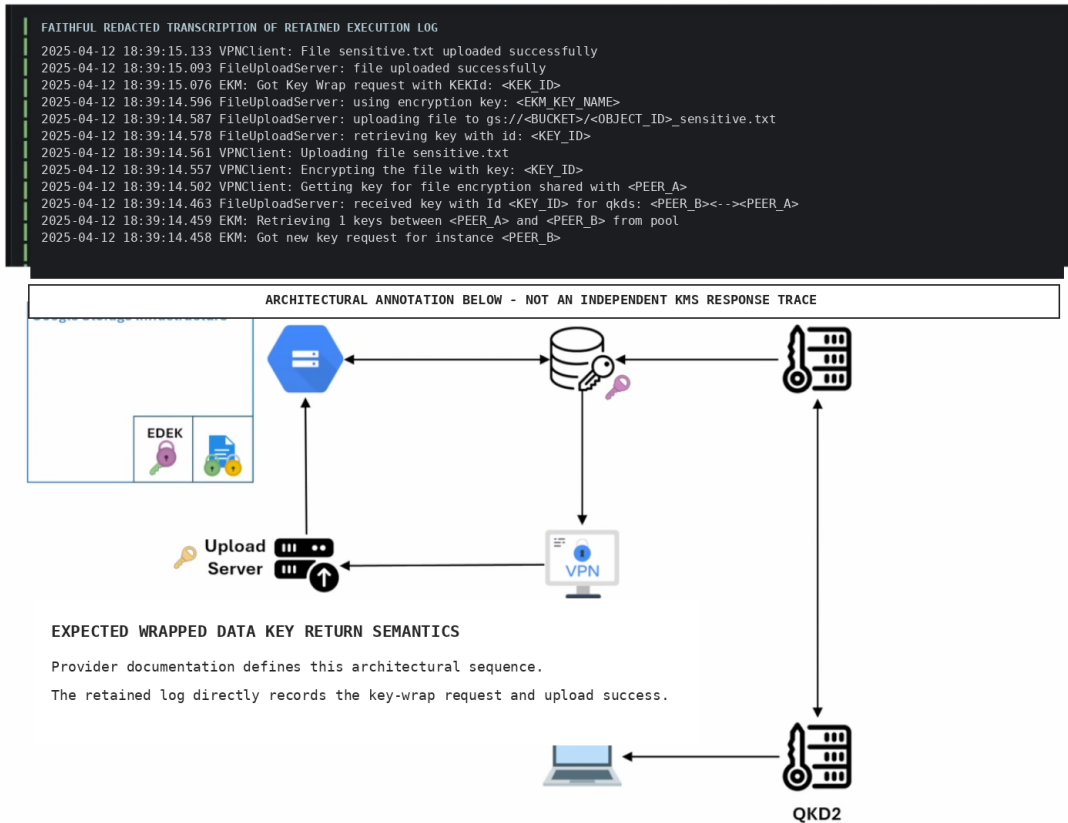


Figure 9: The faithful redacted log records the EKM key wrap request and upload success, while the lower architectural annotation illustrates expected wrapped data key return semantics from provider documentation.

FAITHFUL REDACTED TRANSCRIPTION OF RETAINED TERMINAL CAPTURE

```

<USER>@<HOST>:~$ gcloud storage ls gs://<BUCKET>
gs://<BUCKET>/<OBJECT_ID>_sensitive.txt
<USER>@<HOST>:~$ gcloud storage cp gs://<BUCKET>/<OBJECT_ID>_sensitive.txt ./sensitive.txt
Copying gs://<BUCKET>/<OBJECT_ID>_sensitive.txt to file://./sensitive.txt
  Completed files 1/1 | 48.0B/48.0B
<USER>@<HOST>:~$ cat sensitive.txt
<NON-PLAINTEXT BYTE OUTPUT PRESERVED AS A REDACTED PLACEHOLDER>
<USER>@<HOST>:~$

```

Figure 10: The faithful redacted terminal transcription preserves object listing, transfer, and local inspection after completion of the configured cloud storage workflow.

4. Direct QKD appliance evidence

Figure 11 preserves the original QKD appliance dashboard used as direct measurement provenance. The left panel records secure bit rate, the appliance labeled 256 bit unit rate, signal QBER, signal state values, weak decoy QBER, and weak decoy state values. The right panel records retained secured bits, available 256 bit units, consumed keys, consumed bits, recorded requests, failed requests, maximal key length, generated bits, and deleted bits.

	Param Name	Value		Param Name	Value
②	Secure Bit Rate (bps)	21220.00		Number of available secured bits	838,847,064
②	Secure Key Rate (256bps)	82		Number of available 256 bit keys	3,275,971
②	Signal QBER (%)	0.80		Keys consumed since the last power up	410,188
② >	Signal QBER-per-State (%)	[0.19, 1.04, 1.45, 0.27, 0.83, 0.69]		Bits consumed since the last power up	105,008,128
②	Weak Decoy QBER (%)	1.00		Key requests since the last power up	77,538
② >	Weak Decoy QBER-per-State (%)	[0.48, 1.53, 1.65, 0.69, 1.20, 0.48]		Failed key requests since the last power up	0
				Maximal key length	8,192
				Total generated bits	87,006,129,080
				Number of deleted bits	86,167,282,016

Figure 11: The original appliance dashboard preserves direct rate, channel quality, inventory, request, consumption, generation, and deletion values at the captured operating point.

Table 4 transcribes every appliance value used throughout the manuscript and analytical package. Instantaneous rate and QBER values characterize the captured operating point, while cumulative counters characterize retained totals since the recorded power up interval. The derived unit rate equals the captured secure bit rate divided by 256 and therefore remains a deterministic arithmetic transformation.

The original appliance string “Secure Key Rate (256bps)” remains preserved within Figure 11. The analytical package interprets the associated value as 256 bit allocation units per second because the numerical value aligns closely with 21,220 divided by 256.

Table 4: The transcribed appliance values preserve provenance and distinguish direct observations from deterministic derived quantities.

Metric	Value	Evidence type	Interpretation
Secure bit rate	21,220 bits/s	Direct	Captured secret bit production rate.
Appliance labeled 256 bit unit rate	82 units/s	Direct	Captured appliance rate for 256 bit allocation units.
Derived 256 bit unit rate	82.890625 units/s	Derived	Secure bit rate divided deterministically by 256.
Signal QBER	0.80%	Direct	Captured signal state channel quality.
Weak decoy QBER	1.00%	Direct	Captured weak decoy channel quality.

Metric	Value	Evidence type	Interpretation
Signal QBER per state	0.19, 1.04, 1.45, 0.27, 0.83, 0.69%	Direct	Captured signal error values across six appliance states.
Weak decoy QBER per state	0.48, 1.53, 1.65, 0.69, 1.20, 0.48%	Direct	Captured weak decoy error values across six appliance states.
Available secured bits	838,847,064 bits	Direct	Retained secured inventory at capture time.
Available 256 bit units	3,275,971 units	Direct	Retained allocatable inventory at capture time.
Consumed keys	410,188 units	Direct	Cumulative allocation consumption since recorded power up.
Consumed bits	105,008,128 bits	Direct	Cumulative bit consumption since recorded power up.
Key requests since recorded power up	77,538 requests	Direct	Cumulative appliance request count associated with the retained power up interval.
Failed key requests since recorded power up	0 requests	Direct	Zero appliance side failed key requests recorded within the same power up interval.
Maximal key length	8,192 bits	Direct	Maximum appliance request length.
Total generated bits	87,006,129,080 bits	Direct	Cumulative source generation since recorded power up.
Deleted bits	86,167,282,016 bits	Direct	Cumulative deletion and lifecycle turnover since recorded power up.

Figure 12 reexpresses the twelve captured state QBER values through a grouped analytical chart. The figure supports direct comparison across state indices while retaining the original dashboard as the controlling source record.

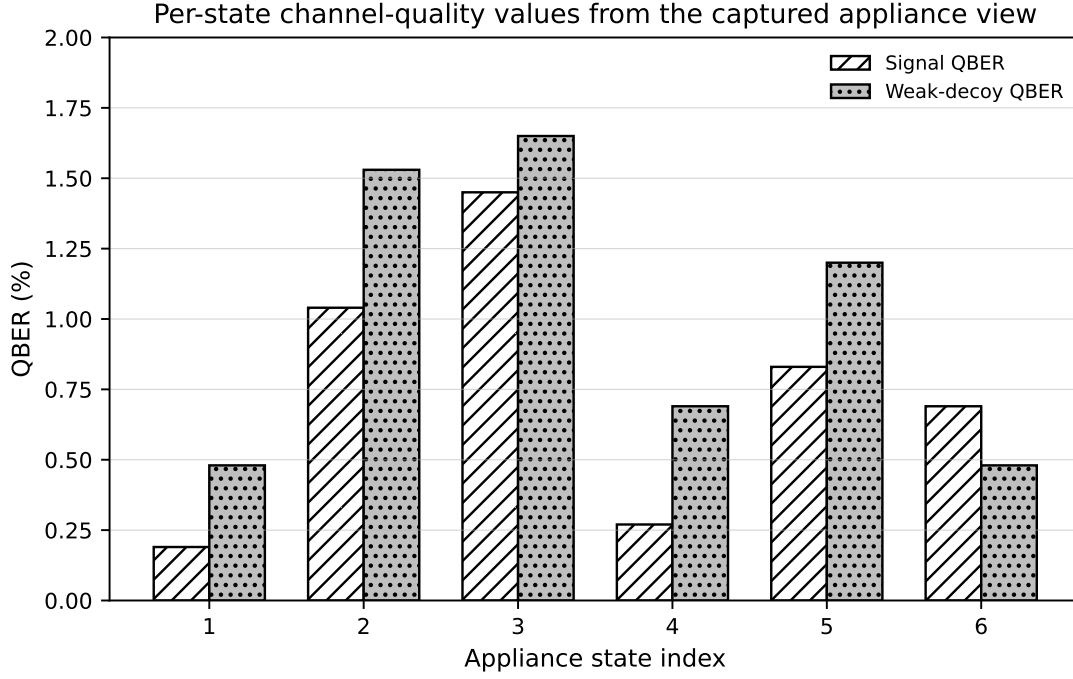


Figure 12: The grouped QBER visualization reexpresses direct signal and weak decoy state values from the captured appliance dashboard.

5. Cryptographic processing evidence profile

Table 5 separates implemented processing evidence, lifecycle semantics, and production conformance parameters. This distinction preserves the exact implementation record while identifying primitive specific evidence required during production acceptance. The lifecycle specification evaluates identifiers, purpose binding, persistent version relationships, and activation freshness independently from a provider selected wrapping primitive. The evaluated profile treats appliance supplied units as opaque 256 bit inputs, while production qualification records key derivation, domain separation, composition, wrapping, and authentication parameters explicitly.

Table 5: The cryptographic processing profile distinguishes implemented evidence, lifecycle semantics, and production conformance parameters.

Processing ment	ele-	Evidence supported by this package	Evidence status	Acceptance interpretation
QKD application delivery		Paired identifiers and appliance supplied 256 bit material enter cloud and client control paths.	Implemented and observed	Supports identifier allocation and matched retrieval across both evaluated workflows.
External key binding		Consumed identifiers receive persistent EKM version relationships after allocation.	Implemented and specified	Supports custody traceability, non reuse, and deterministic version resolution.
VPN purpose binding		Each allocated identifier receives one VPN activation relationship before coordinated use.	Implemented and specified	Supports purpose separation and freshness around WireGuard PSK updates.
Interface authentication		KMS facing and VPN facing calls use configured JWT paths, while client ingress uses identity aware mediation.	Implemented and con-figuration	Supports authorised control path participation within the prototype trust model.
Prototype AES component	local	Retained implementation records identify an AES 256 processing component within the external service domain.	Component level evidence	Supports service local processing while production wrapping conformance uses a dedicated parameter profile.
Production primitive profile	primitive	Approved wrapping mode, nonce construction, integrity verification, error semantics, and interoperability testing receive deployment specific documentation.	Qualification parameter	Supports primitive specific assurance while retaining the portable lifecycle semantics evaluated by the article.

6. Executable lifecycle validation evidence

The executable reference specification represents source availability, EKM reachability, temporary inventory, persistent bindings, active EKM version, pending VPN activation, active VPN association, activation epoch, continuity authority, recovery reconciliation, one derived lifecycle mode, and three derived service consequence indicators. Its finite domains contain four key identifiers, two external key versions, three activation epochs, and every enabled availability, allocation, policy, and recovery event defined by the specification. Identifier labels remain behaviorally symmetric, while four identifiers support simultaneous active EKM, active VPN, pending VPN, and available inventory relationships. Two external key versions exercise version replacement, while three ordered activation epochs exercise initial activation, later refresh, and stale replay rejection.

Complete breadth first exploration continued until the reachable state closure stabilized within the declared finite domains. The completed graph contains 45,824 reachable states, 307,680 labeled transitions, and a maximum shortest path depth of sixteen events. The exploration covers ingestion, expiration, EKM allocation, VPN allocation, VPN activation,

stale epoch rejection, source interruption and restoration, EKM interruption and restoration, continuity authority changes, suspension, and recovery reconciliation. Four transition guards independently verify ingestion freshness, allocation prerequisites, stale activation rejection, and recovery reconciliation prerequisites.

Table 6: The executable specification evaluates nine state properties throughout the complete reachable state closure.

ID	Checked state property	Exploration result
I1	Available identifiers remain unique throughout every temporary pool transition.	Preserved across all reachable states.
I2	Every consumed identifier appears within at most one persistent purpose binding.	Preserved across all reachable states.
I3	Consumed identifiers remain excluded from every subsequent available allocation pool.	Preserved across all reachable states.
I4	Every purpose and object pair resolves toward at most one identifier.	Preserved across all reachable states.
I5	Every active EKM version resolves toward its corresponding persistent identifier binding.	Preserved across all reachable states.
I6	Pending VPN activation retains a matching binding and strictly increasing activation epoch.	Preserved across all reachable states.
I7	Active VPN state matches its persistent binding and authoritative activation epoch.	Preserved across all reachable states.
I8	Every modeled condition resolves toward one lifecycle mode derived from dependencies and policy.	Preserved across all reachable states.
I9	Storage path, fresh allocation, and established VPN consequence indicators match modeled dependencies.	Preserved across all reachable states.

A separate sensitivity analysis seeded one deliberately inconsistent state for each principal property and one inconsistent transition for each guard family. All nine invalid states and all four invalid transitions triggered at least one corresponding check, demonstrating that the validation logic detects representative inconsistencies independently from the valid transition generator.

Table 7: The seeded sensitivity cases confirm representative inconsistency detection across nine state properties and four transition guard families.

ID	Seeded inconsistency	Expected detection
I1	Duplicate identifier appears inside the available allocation pool.	Duplicate inventory assertion detects the condition.
I2	One identifier appears inside two persistent purpose bindings.	Identifier binding uniqueness assertion detects the condition.
I3	Consumed identifier appears again inside the available allocation pool.	Pool and binding intersection assertion detects the condition.
I4	Two identifiers resolve toward the same purpose and object pair.	Purpose object uniqueness assertion detects the condition.
I5	Active EKM version lacks a corresponding persistent identifier relationship.	Active EKM binding assertion detects the condition.
I6	Pending VPN activation carries an epoch that is stale or replayed.	Pending activation freshness assertion detects the condition.
I7	Active VPN epoch conflicts with the authoritative lifecycle epoch.	Active VPN binding assertion detects the condition.
I8	Stored lifecycle mode conflicts with the mode derived from dependency state.	Lifecycle mode derivation assertion detects the condition.
I9	Stored service consequence indicator conflicts with current dependency state.	Service consequence derivation assertion detects the condition.
G1	Ingestion proceeds while the QKD source remains unavailable.	Source availability guard detects the invalid transition.
G2	Fresh EKM allocation proceeds while the EKM remains unavailable.	Allocation prerequisite guard detects the invalid transition.
G3	A stale VPN rejection event changes authoritative lifecycle state.	State preserving rejection guard detects the invalid transition.
G4	Recovery reconciliation proceeds before restored dependencies and pending recovery exist.	Recovery prerequisite guard detects the invalid transition.

Table 8 connects each model element with its source basis and evidentiary classification. The mapping distinguishes implemented prototype behavior from executable abstractions introduced for explicit lifecycle governance and validation.

Table 8: The model traceability matrix distinguishes implemented behavior, source observations, provider semantics, and executable lifecycle abstractions.

Model element	Source basis	Classification	Interpretive role
QKD source availability	Appliance source state and qualitatively observed interruption of new input during the prototype campaign.	Observed and specified	Governs ingestion availability while retained inventory remains independently represented.
EKM reachability	Implemented external service path together with provider documented dependency behavior for managed operations.	Implemented and reasoned	Separates binding retention from managed service access consequences.
Temporary inventory and persistence	EKM decomposition, temporary pool, TTL cleanup, and retained consumed identifier relationships.	Implemented	Supports allocation, expiration, persistent binding, and non reuse properties.
Purpose and object bindings	EKM version relationships and VPN session allocation logic preserved within implementation records and algorithms.	Implemented and specified	Assigns every consumed identifier toward one authorised lifecycle purpose and object.
VPN activation epoch	Prototype identifier and effective time exchange, extended through an ordered epoch abstraction inside the executable specification.	Observed and modelled	Supports freshness checking, stale activation rejection, and deterministic active state.
Lifecycle modes	Buffered continuity and binding retention observations combined with explicit exhaustion, recovery, and policy authority rules.	Observed and specified	Provides one operator facing governance state across dependency changes.
Service consequence indicators	Provider EKM semantics, implemented VPN state, and the declared dependency interpretation profile.	Reasoned and derived	Distinguishes storage path availability, fresh allocation capability, and established VPN continuity.

The reference specification stores identifiers and lifecycle state rather than secret key material. This design supports complete state exploration while preserving the evidence release boundary. The package contains the Python specification, regression tests, generated JSON and Markdown reports, complete state table, complete labeled transition table, and seeded state and transition sensitivity register.

7. Analytical source capacity and continuity evidence

The analytical model uses captured source rate $r = 82.890625$ units per second, retained inventory $B = 3,275,971$ units, external key rotation interval $\tau_E = 900$ seconds, VPN refresh interval $\tau_V = 3600$ seconds, and one 256 bit unit per configured lifecycle event. A source allocation fraction α represents the share of captured production assigned toward these two lifecycles, while an inventory allocation fraction β represents the share of captured retained inventory reserved for continuity analysis.

The configured pair demand equals

$$D_0 = \frac{1}{900} + \frac{1}{3600} = 0.001388889 \text{ units per second.} \quad (1)$$

One equivalent lifecycle pair denotes one external key version rotation stream and one VPN refresh stream operating at the declared cadences. The corresponding QKD source side ceiling equals

$$m_Q = \frac{\alpha r}{D_0}, \quad (2)$$

which produces 59,681.25 equivalent lifecycle pairs when $\alpha = 1$ under the declared allocation abstraction. This value represents a source feasibility ceiling rather than measured end to end service scalability.

Actual operational scale follows the smallest measured capacity across relevant system dependencies:

$$m_{op} = \min(m_Q, m_E, m_V, m_C, m_P), \quad (3)$$

where m_E , m_V , m_C , and m_P represent measured EKM, VPN, cloud service, and policy capacity respectively. The available evidence quantifies m_Q , while deployment qualification supplies the remaining terms after topology, workload, and service objectives become fixed.

Table 9: The analytical scenarios derive demand, source headroom, interruption endurance, and refill behavior from captured source values and configured cadence.

Equivalent pairs	Demand	Source headroom	Endurance hours	Refill minutes	Analytical interpretation
1	0.001389	59,681.25	655,194.2	0.001	Configured prototype pair remains deeply inside captured source capacity.
1,000	1.388889	59.681	655.194	1.022	Equivalent source demand retains substantial production margin.
10,000	13.888889	5.968	65.519	12.078	Equivalent source demand remains sustainable with reduced refill margin.
50,000	69.444444	1.194	13.104	309.60	Equivalent source demand approaches the captured production boundary.
60,000	83.333333	0.995	10.920	—	Equivalent source demand exceeds captured production while demand persists.

Figure 13 maps analytical refill duration across source utilization and interruption duration after source production returns. Curved contours show modest refill burden throughout lower utilization regions and rapidly increasing restoration effort as lifecycle demand approaches captured production.

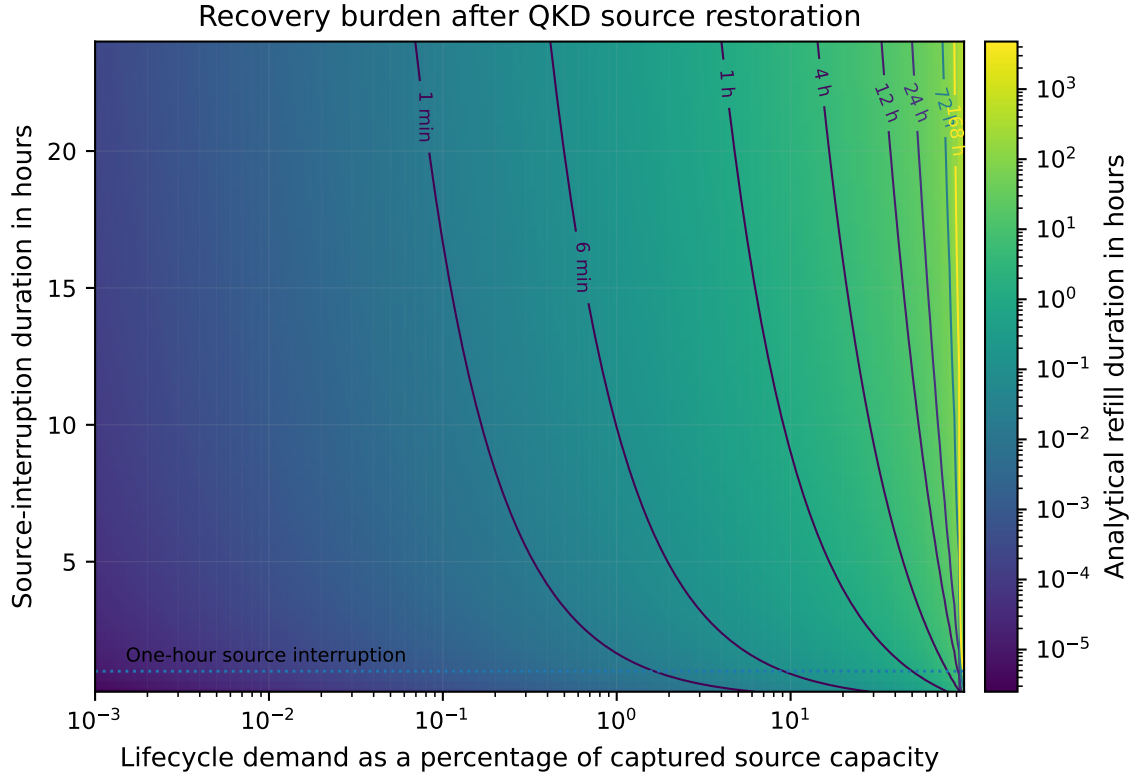


Figure 13: The analytical recovery map shows refill duration after source restoration across lifecycle utilization and interruption duration under declared assumptions.

8. Continuity evidence and service consequence interpretation

The prototype campaign recorded two qualitative continuity behaviors that inform the lifecycle specification. During interruption of new QKD input, retained inventory continued supporting control path allocation. During temporary disruption affecting fresh lifecycle operations, duration management preserved the current authoritative binding. These observations support buffered continuity and binding holdover semantics, while actual managed storage access remains governed by EKM reachability.

Controlled exhaustion, interruption endurance, and refill duration receive executable and analytical interpretation within the current package. The executable specification defines deterministic state changes, while equations quantify source dependent bounds under declared assumptions. This combination supports lifecycle assurance conclusions while classifying analytical outputs separately from timed failure recovery distributions.

Table 10: The continuity evidence register distinguishes qualitative observations from specified and analytically characterized behavior.

Condition	Evidence and lifecycle response	Supported interpretation
QKD source interruption with retained inventory	Qualitative observation recorded paused ingestion while retained inventory continued supporting allocation.	Buffered continuity operated while reserved inventory remained available.

Condition	Evidence and lifecycle response	Supported interpretation
Source interruption after reserved inventory depletion	Executable specification enters binding holdover, controlled exhaustion, or policy suspension according to active state and authority.	New allocation follows explicit policy while authoritative active state remains unambiguous.
Temporary EKM unavailability	Lifecycle specification preserves authoritative bindings while reporting managed storage dependency blockage and limited fresh allocation.	Binding retention and managed storage availability remain distinct operational properties.
Established VPN during dependency change	Existing VPN state can remain active while refresh and new session allocation follow current dependency and policy state.	Established tunnel continuity remains distinct from allocation and managed storage consequences.
Dependency restoration	Recovery reconciliation checks restored dependencies, retained bindings, inventory, active state, and continuity authority.	Return toward ready operation follows one deterministic governance sequence.

9. Claim evidence mapping

Table 11 maps each principal manuscript conclusion toward its controlling evidence and interpretation boundary. The mapping prevents direct device observations from supporting cloud performance claims and prevents equations from becoming observed workload behavior.

Table 11: The claim evidence register connects principal conclusions with controlling records and explicit interpretation boundaries.

ID	Principal claim	Controlling evidence	Interpretation boundary
C1	The external KEK and EKM layer preserves provider generated DEK responsibility while externalising custody.	Provider semantics, boundary comparison, and managed storage workflow records.	Supports architecture and responsibility separation for equivalent external custody models.
C2	The lifecycle supports persistent version binding, temporary pooling, purpose separation, and controlled continuity.	EKM decomposition, lifecycle procedure, executable specification, and retained identifier records.	Supports implemented and specified behavior within the prototype case and declared model.
C3	QKD identifiers support coordinated WireGuard PSK activation within the implemented access path.	VPN trace, connected state, topology, and lifecycle procedure.	Supports functional control plane coordination while retaining WireGuard endpoint assumptions.
C4	The captured source produced 21,220 secure bits per second and retained 3,275,971 available 256 bit units.	Original appliance dashboard and transcribed observation table.	Supports the captured operating point rather than longitudinal source stability.

ID	Principal claim	Controlling evidence	Interpretation boundary
C5	The configured lifecycle pair consumes 0.001388889 256 bit units per second.	Configured intervals and deterministic demand equation.	Supports the declared event abstraction and configured policy cadence.
C6	The captured rate provides a QKD source side ceiling of 59,681.25 equivalent lifecycle pairs when $\alpha = 1$.	Direct rate, derived unit rate, and source ceiling equation.	Supports source feasibility rather than measured EKM or cloud service scalability.
C7	Buffered continuity and debinding holdover provide defined responses during dependency changes.	Qualitative observations, service consequence matrix, and executable lifecycle specification.	Supports observed modes and deterministic governance semantics.
C8	The executable lifecycle preserves nine state properties and four transition guards throughout the complete reachable graph.	Complete state closure, labeled transition table, guard checks, regression tests, and assertion sensitivity records.	Supports internal model consistency rather than deployed cryptographic security.
C9	The model can transfer toward providers exposing equivalent custody, versioning, authentication, observability, and continuity semantics.	Provider neutral conditions, standards aligned interfaces, and parameterized analytical model.	Supports architectural transfer analysis rather than empirical multicloud replication.

10. Reproducibility and redaction rules

The reproducibility package contains structured CSV files for the appliance observation, state QBER values, analytical scenarios, capacity surfaces, continuity surfaces, evidence register, complete lifecycle states, complete labeled transitions, and seeded state and transition sensitivity cases. The package also contains the executable lifecycle specification, regression tests, generated JSON and Markdown reports, figure generation code, Graphviz sources, language audits, and compilation instructions.

Every released record follows explicit confidentiality controls protecting operational and cryptographic information. Public tables, logs, image annotations, and model states preserve lifecycle relationships while excluding secret key material. Released data excludes credentials, reusable tokens, private addresses, and sensitive infrastructure identifiers. Faithful redacted transcriptions preserve the original line order and timestamps while replacing operational identifiers with explicit placeholders. Lower storage diagrams remain visibly labeled as architectural annotations, which separates provider semantic interpretation from directly retained execution events.

The main manuscript and supplementary source compile through the Elsevier `elsarticle` class. Figure generation uses Python together with pandas, NumPy, matplotlib, Graphviz, and Pillow dependencies. Generative AI assistance supported preliminary scaffolding for selected reproducibility scripts and explanatory diagram sources. The authors revised and executed every script, while the redaction workflow preserved retained event order and timestamps through explicit transcriptions without adding execution events. The project README records commands, directory structure, evidence provenance, redaction rules, tool assistance, and source preservation procedures.

11. Summary

The supplementary package provides a traceable evidence chain supporting the revised claim set. Executed workflow records establish functional sequence across VPN and managed storage control paths. Direct appliance values establish one captured source operating point and cumulative device history. Complete reachable state exploration establishes internal consistency throughout the declared finite state domains. Deterministic equations establish source capacity, interruption endurance, depletion onset, refill behavior, and transfer qualification conditions.

The combined evidence supports boundary correctness, lifecycle consistency, service specific continuity interpretation, functional realization, captured source feasibility, and provider neutral transfer conditions. Deployment specific latency, throughput, concurrent demand, and timed recovery measurements remain qualification variables because their values depend upon selected topology, workload, implementation resources, and service objectives. The manuscript therefore preserves strong conclusions within the available evidence while defining a precise pathway toward subsequent operational qualification.